



THUẬT TOÁN QUAY LUI



Nội dung:

- 1. Giới thiệu
- 2. Mã giả
- 3. Quay lui xâu nhị phân
- 4. Quay lui hợp chập K của N phần tử
- 5. Quay lui hoán vị
- 6. Quay lui phân hoạch
- 7. Bài toán N Queen
- 8. Người du lịch

1. Giới thiệu:



Thuật toán quay lui là một phương pháp vét cạn dùng với các bài toán liệt kê hoặc đếm cấu hình, thuật toán có ý tưởng là xây dựng dần các cấu hình

Giả thiết cấu hình của bài toán cần tìm được mô tả bằng một bộ gồm N thành phần $X[1], X[2], X[3] \dots X[N]$

Giả sử đã xác định được $i - 1$ thành phần $X[1], X[2], \dots X[i - 1]$, bây giờ ta sẽ xác định thành phần $X[i]$ bằng cách xét tất cả các khả năng (N_i) có thể nhận của $X[i]$. Với khả năng này ký hiệu là j có thể xảy ra 2 trường hợp

- Nếu chấp nhận j thì $X[i] = j$, sau đó nếu $i = n$ tức đã xây dựng xong cấu hình, ngược lại thì tiếp tục gọi đệ quy để đi tìm $X[i + 1]$.
- Nếu thử tất cả các khả năng của j mà không có khả năng nào chấp nhận được thì quay lại bước trước để xác định lại $X[i - 1]$.

Điểm quan trọng của thuật toán là phải ghi nhớ tại mỗi bước đã đi qua, những khả năng nào đã thử để tránh trùng lặp



2. Mã giả:



Mã giả:

```
Try (int i ) {  
    for ( j = <Khả năng 1>; j <= ni; j++ ){  
        if (<chấp thuận khả năng j> ) {  
            X[i] = <khả năng j>;  
            if ( i ==n)  
                Result();  
            else  
                Back_Track(i+1);  
        }  
    }  
}
```

3. Quay lui xâu nhị phân:

Phân tích

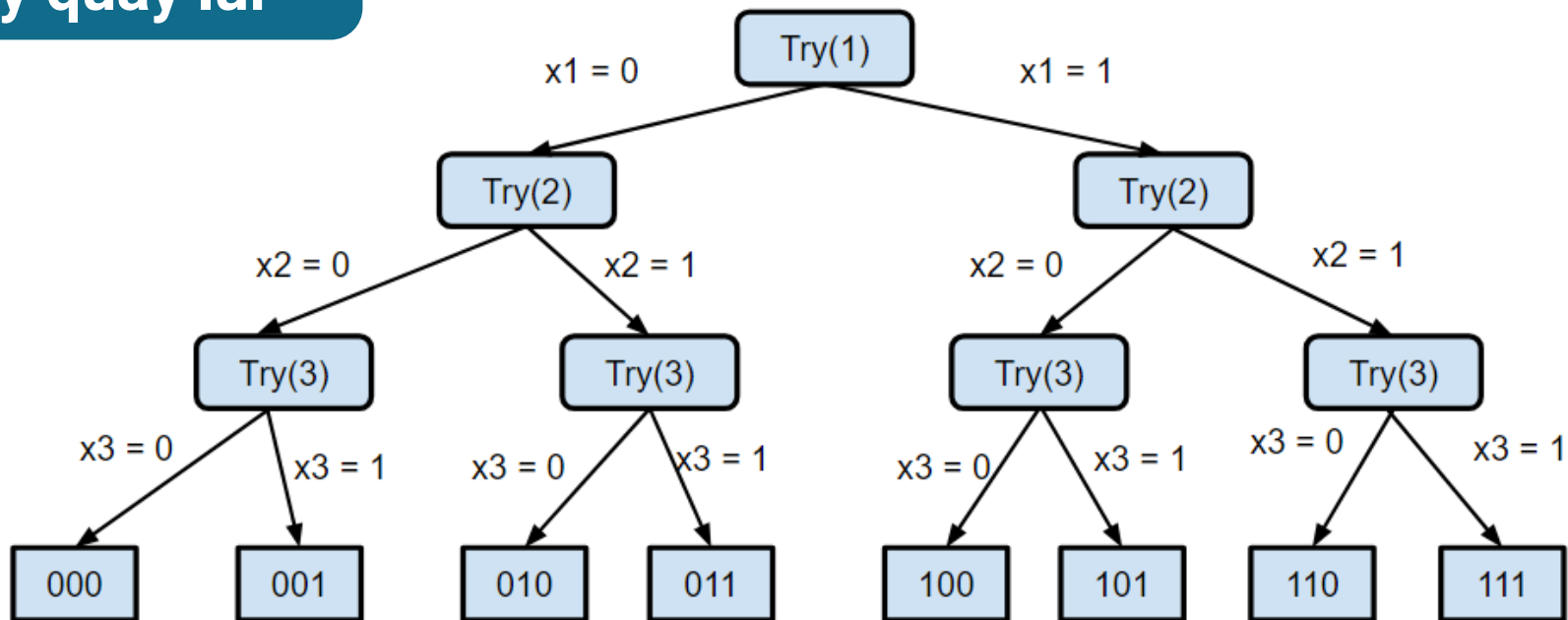
Đối với quay lui xâu nhị phân thì tại $X[i]$ chỉ có thể nhận giá trị 0 hoặc 1. Trong hàm main ta chỉ cần gọi lời gọi **Try(1)**.
Hàm **Result()** là hàm in ra cấu hình xâu nhị phân

Hàm quay lui

```
void Try(int i){  
    for(int j = 0; j <= 1; j++){  
        X[i] = j;  
        if(i == n)  
            Result();  
        else  
            Try(i + 1);  
    }  
}
```

3. Quay lui xâu nhị phân:

Cây quay lui



28tech.com.vn

4. Quay lui tổ hợp chập K của N phần tử:

Phân tích

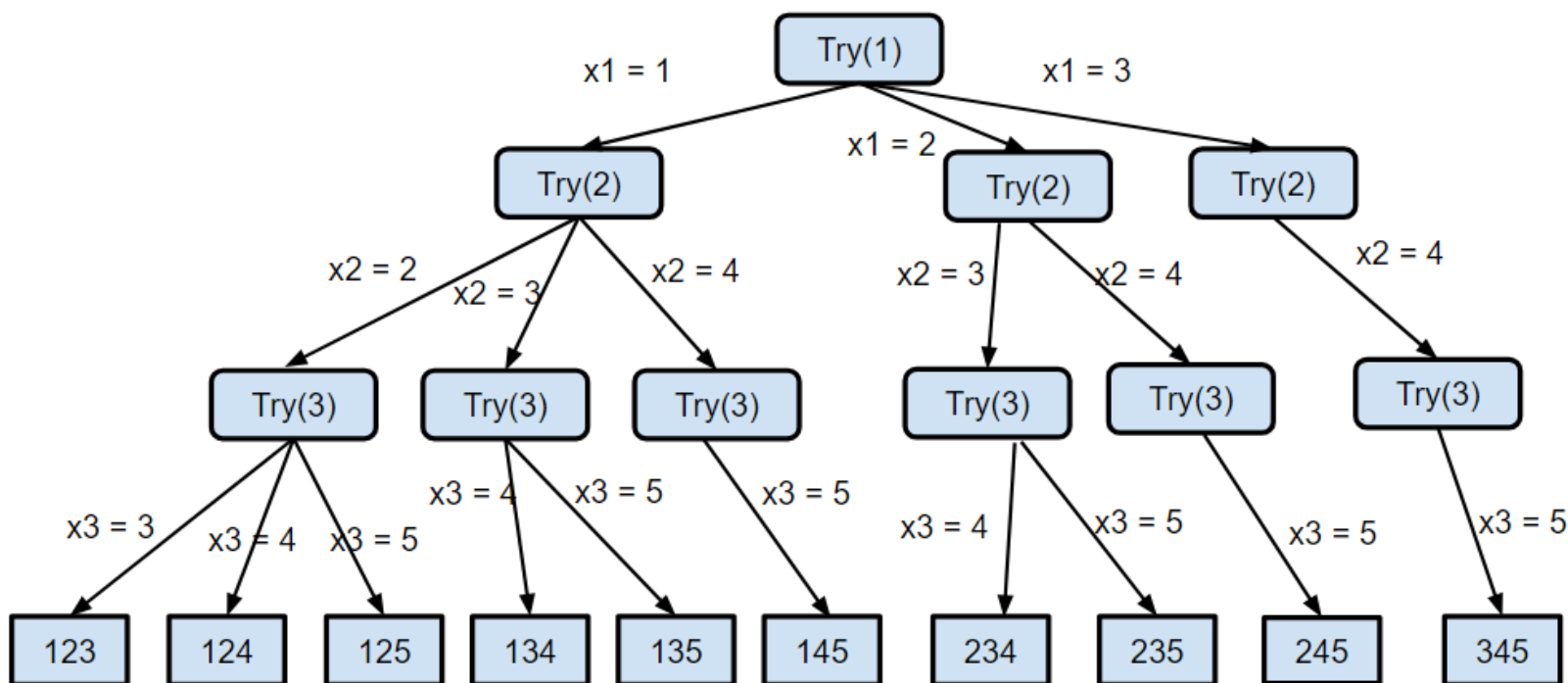
Đối với quay lui sinh tổ hợp thì $X[i]$ tối thiểu phải lớn hơn phần tử đứng trước nó trong cấu hình là $X[i - 1]$, giá trị tối đa $X[i]$ có thể nhận là $n - k + i$. Trong main ta chỉ cần gọi $\text{Try}(1)$ là có thể sinh ra mọi cấu hình.

Hàm quay lui

```
void Try(int i){  
    for(int j = X[i - 1] + 1; j <= n - k + i; j++){  
        X[i] = j;  
        if(i == k)  
            Result();  
        else  
            Try(i + 1);  
    }  
}
```

4. Quay lui tổ hợp chập K của N phần tử:

Cây quay lui



28tech.com.vn

5. Quay lui hoán vị

Phân tích

$X[i]$ có thể nhận bất kỳ giá trị nào trong khoảng từ 1 từ n với điều kiện nó chưa được dùng để gán cho các phần tử từ $X[1], X[2]...X[i - 1]$.

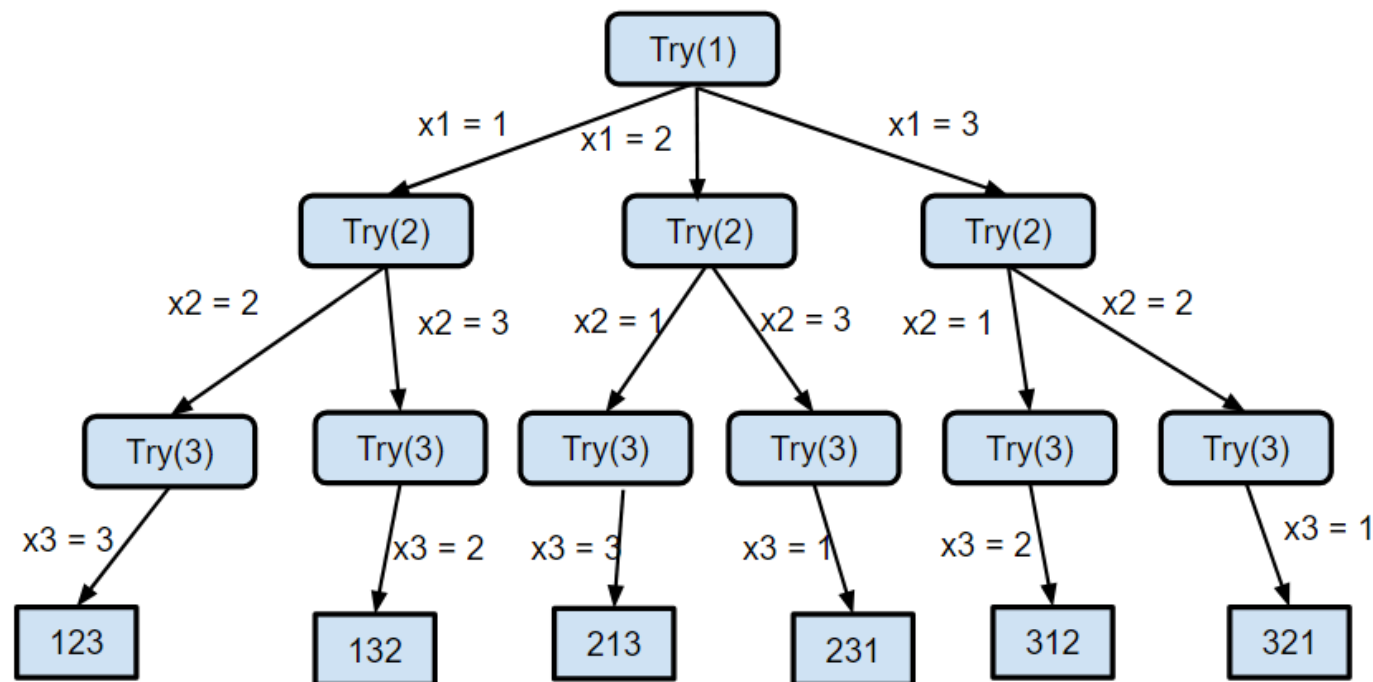
Ở đây ta dùng mảng `used[]` để đánh dấu các giá trị đã được sử dụng. Trong main sẽ gọi đệ quy `Try(1)`, mảng `used[]` cần được khởi tạo là các số 0

Hàm quay lui

```
void Try(int i){
    for(int j = 1; j <= n; j++){
        if(used[j] == 0){
            X[i] = j;
            used[j] = 1;
            if(i == n) Result();
            else Try(i + 1);
            used[j] = 0;
        }
    }
}
```

5. Quay lui hoán vị:

Cây quay lui



28tech.com.vn



6. Quay lui phân hoạch



Trong main ta gọi `Try(1, 0, n)` để quay lui ra các phân hoạch

```
void Try(int i, int sum, int init){
    for(int j = init; j >= 1; j--){
        X[i] = j;
        sum += j;
        if(sum == n){
            for(int k = 1; k <= i; k++){
                cout << X[k] << " ";
            }
            cout << endl;
        }
        else if(sum < n){
            Try(i + 1, sum, j);
        }
        sum -= j;
    }
}
```



7. Bài toán N Queen



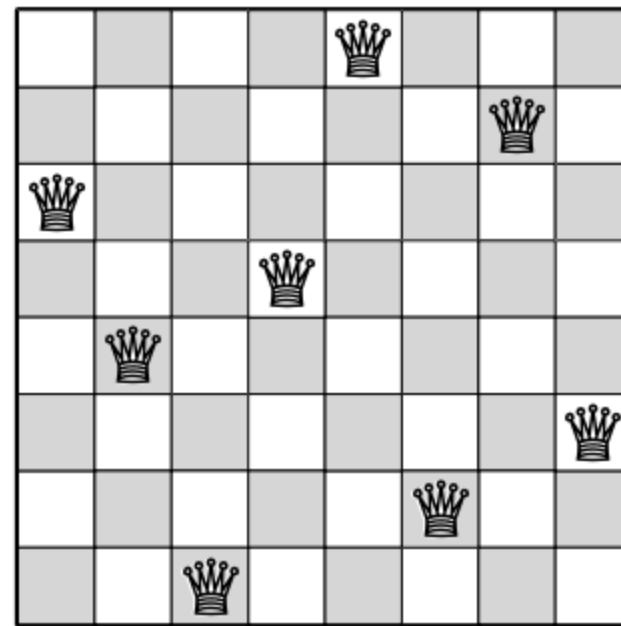
Bài toán N Quân Hậu

Cho bàn cờ vua cỡ $N \times N$, hãy tìm các cách đặt N quân hậu vào bàn cờ sao cho không có 2 con hậu nào ăn nhau, con hậu có thể ăn theo hàng ngang, dọc hoặc đường chéo.

Gọi $X[1], X[2], \dots, X[N]$ là một nghiệm của bài toán, khi đó $X[i] = j$ có nghĩa là ta đặt con hậu ở hàng i nằm ở cột j . Như vậy sẽ loại bỏ được việc 2 con hậu ở cùng 1 hàng, ta chỉ cần xử lý cột và đường chéo.

Cấu hình bên phải tương đương với $X[1] = 5, X[2] = 7, X[3] = 1, X[4] = 4, X[5] = 2, X[6] = 8, X[7] = 6, X[8] = 3$.

Ví dụ về 1 cách xếp hợp lệ

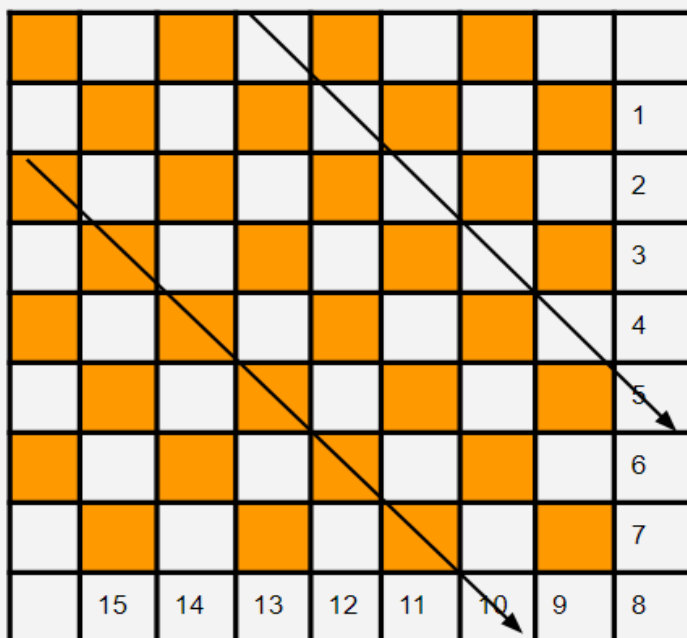


7. Bài toán N Queen

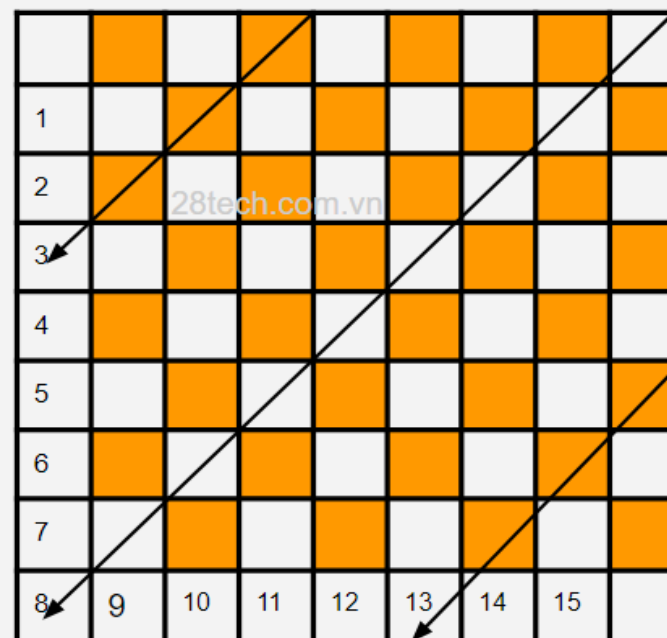


Để quản lý các cột, đường chéo còn trống ta dùng 3 mảng `cot[]`, `xuoi[]`, `nguoc[]`. Với bàn cờ cỡ $N \times N$ sẽ có N cột, $2N - 1$ đường chéo xuôi và $2N - 1$ đường chéo ngược.

Số thứ tự đường chéo xuôi



Số thứ tự đường chéo ngược

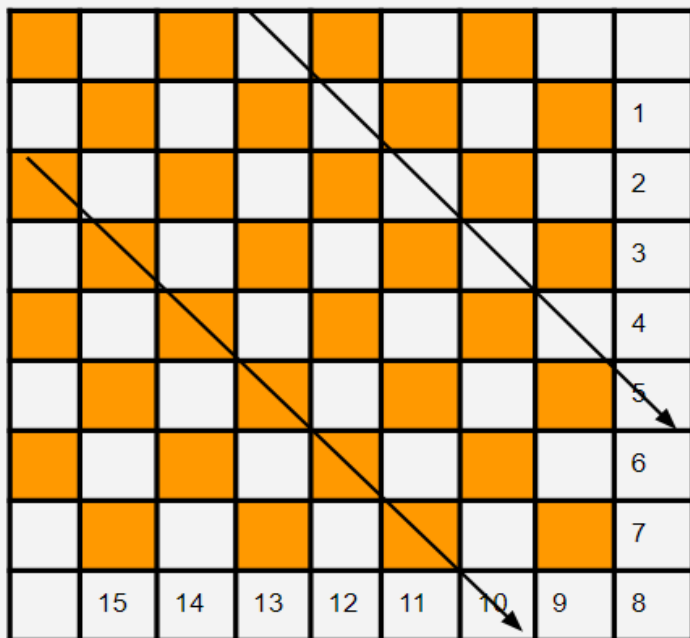


7. Bài toán N Queen

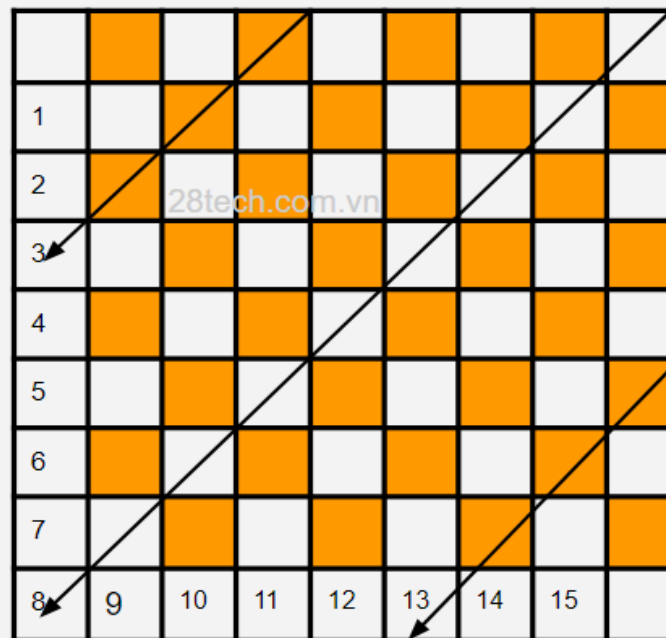


Khi gán $X[i] = j$ cần kiểm tra hàng j , đường chéo xuôi thứ $i - j + n$ và đường chéo ngược $i + j - 1$

Xuoi[$i - j + n$]



Ngucoc[$i + j - 1$]



7. Bài toán N Queen



Mã nguồn

```
int n, X[100], cot[100], xuoi[100], nguoc[100];

void Try(int i){
    for(int j = 1; j <= n; j++){
        if(cot[j] == 0 && xuoi[i - j + n] == 0 && nguoc[i + j - 1] == 0){
            X[i] = j;
            cot[j] = xuoi[i - j + n] = nguoc[i + j - 1] = 1;
            if(i == n){
                //result
                ++dem;
            }
            else
                Try(i + 1);
            //backtrack
            cot[j] = xuoi[i - j + n] = nguoc[i + j - 1] = 0;
        }
    }
}
```

Ban đầu các mảng cot[], xuoi[], nguoc[] đều có giá trị là 0 tương ứng với còn trống
Để giải bài toán ta chỉ cần gọi hàm Try(1)

